01 — AML PROJECT · PHASE 1

REMAINING USEFUL LIFE OF TURBOFAN ENGINES

Predicting how many flights an engine has left from its sensor history, on NASA's C-MAPSS data (subset FD001).

TEAM

Gokul VKS · Milind Bansal · Isha Tomar · Yashi Agrawal · Mohammad Affan Anas

NEWTON SCHOOL OF TECHNOLOGY · RISHIHOOD UNIVERSITY

Predict failure before it happens

Every flight (one **cycle**) wears an engine a little, and its sensors drift as it wears. We predict its **remaining useful life (RUL)**: the cycles it has left before failure.

RUL = LAST CYCLE − CURRENT CYCLE

Training engines run all the way to failure. Test engines stop early; NASA gives their true RUL at the last recorded cycle.

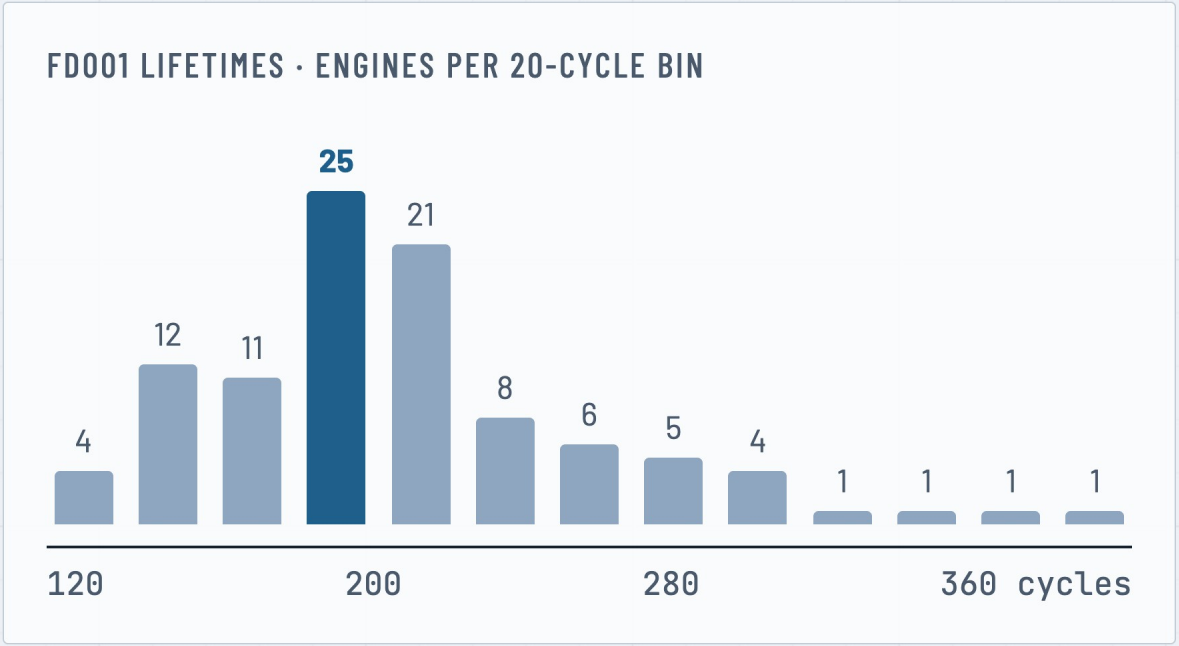
100
training engines, run to failure

100
test engines, scored at their last cycle

21 + 3
sensors and operating settings per cycle

128–362
cycles lived by training engines (median 199)

Engines age differently, so we split by engine



6 **constant sensors** in FD001 carry no signal, so they are dropped.

6 **operating regimes** in FD002/FD004 shift sensor levels, so sensors are normalised per regime.

s_11 **drifts slowly, then faster** near failure, so we add rolling means and slopes.

Cycles of one engine are near-copies, so every split keeps an engine whole (GroupKFold on engine ID).

Three steps: clean, normalise, roll

01

Clean

Drop sensors that never change in the training fold: 21 → 15 in FD001.

Isha Tomar · cleaning.py

02

Normalise

Find operating regimes with k-means, then z-score each sensor within its regime.

Milind Bansal · regimes.py

03

Roll

Over the last 5, 10 and 20 cycles: mean (level), slope (speed of drift), health (change since new).

Isha Tomar · rolling.py

155 predictors per cycle, all backward-looking. Fitted on training engines only, and refitted inside every cross-validation fold.

Three models, one fair protocol

Random Forest

Many trees on random samples, averaged. Lowers variance and catches non-linear sensor interactions.

`Chosen: 100 trees · min leaf 1`

XGBoost

Trees added one at a time, each fitting the errors left by the last (gradient boosting). Lowers bias.

`Chosen: 200 trees · depth 5`

Elastic Net

Linear regression with L1 + L2 penalties that keep correlated sensors together. A readable reference.

`Chosen: alpha 1.0 · L1 ratio 0.2`

SPLIT	TUNE	TARGET	SCORE
By engine, never by row	3-fold engine-grouped CV, seed 42	Uncapped RUL	RMSE, MAE, NASA score, by stage

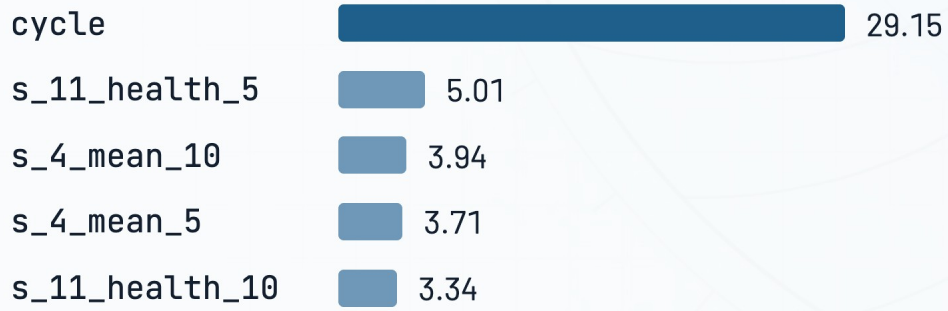
A fair comparison, then what drives it

RMSE · RAW vs ENGINEERED · 80 / 20 ENGINES

	MEAN CV		HOLDOUT	
Model	**Raw**	**Eng.**	**Raw**	**Eng.**
Rand. Forest	39.16	39.27	30.76	26.47
XGBoost	39.89	40.87	30.90	30.33
Elastic Net	42.65	42.88	31.74	33.63

Preprocessing is refitted inside every fold. Engineered features help the trees on this holdout, but **no model improves on mean CV RMSE**, so one split proves nothing.

TREE SHAP · ENGINEERED RF · MEAN |SHAP|



Cycles of predicted RUL, on 200 held-out cycles; additivity checked. It explains the model, not engine physics.

Engineered features: 27.55 RMSE on the test set

	ENGINEERED · PHASE 1 METHOD			RAW INPUTS · COMPARISON		
Model	**RMSE**	**MAE**	**NASA**	**RMSE**	**MAE**	**NASA**
Random Forest	27.55	17.67 ★	145,410	26.15 ★	19.22	8,414
XGBoost	30.50	19.82	61,202	26.84	19.19	7,002
Elastic Net	28.39	23.57	7,777	31.08	25.80	6,903 ★

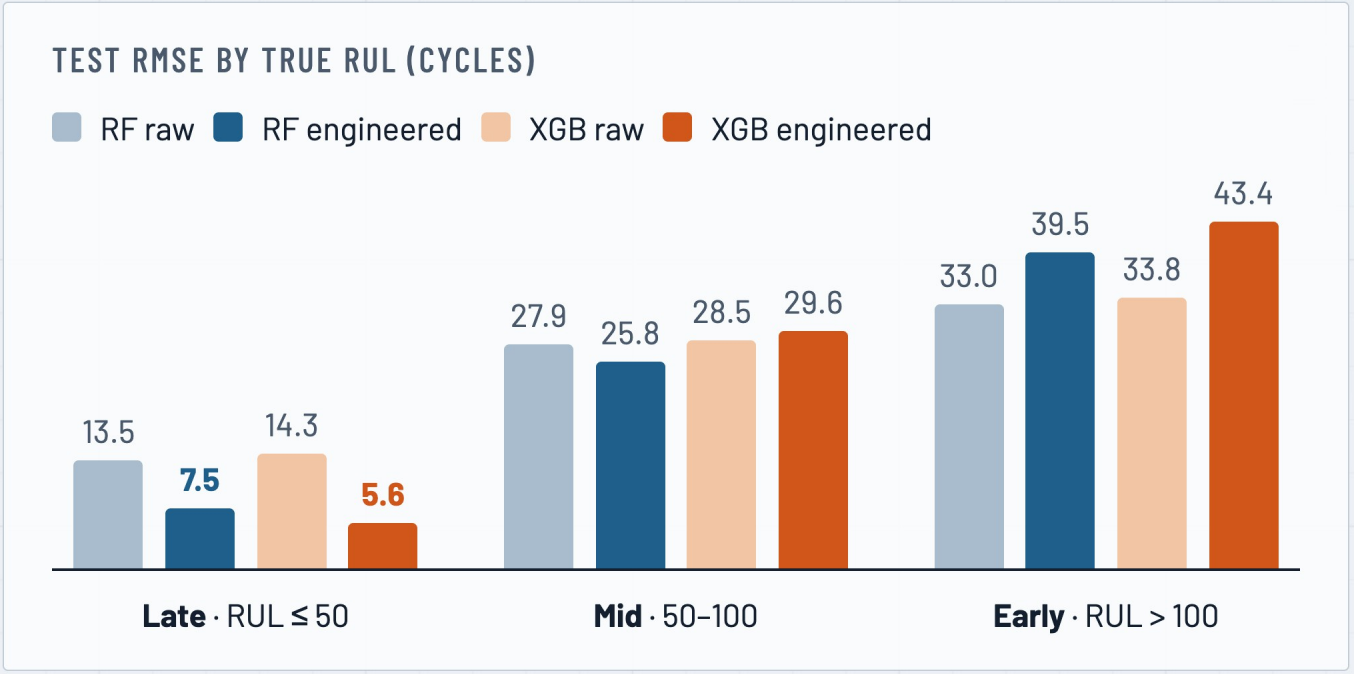
100 test engines, each scored once at its last recorded cycle. Lower is better. ★ = best of the six models.

27.55 RMSE of the Phase 1 method, engineered Random Forest

17.67 MAE, the best of all six models

26.15 RMSE with raw inputs, for comparison

Strong near failure, weak on young engines



7.52

Late-stage RMSE for engineered RF (raw: 13.50), where maintenance gets scheduled.

92%

of engineered RF's NASA score comes from one young engine: 112 cycles left, predicted 230.

Next: learn from the raw sequence

2A · Sequence

LSTM, temporal CNN and Transformer on fixed-length sensor windows, with the same engine splits and metrics.

2B · Hybrid

A sequence encoder plus our engineered-feature branch, fused into one regression head.

FOLLOW-UPS · ABLATIONS AND VALIDATION

A capped-target ablation tests whether the raw-vs-engineered conclusion depends on how RUL is defined. A first Random Forest check already reverses it (16.21 engineered vs 18.66 raw, backup A7). Plus wider tuning, confidence intervals and feature ablations.

BAR TO BEAT

27.55 RMSE for the Phase 1 method (raw inputs: 26.15), and ≈ 6–8 RMSE near failure.

QUESTIONS?

Same splits, same metrics, same engines.
Phase 2 starts from a fair baseline.

Gokul VKS
Lead · data, labels, metrics, test results, literature

Milind Bansal
Splits, EDA, regime normalisation, comparison, SHAP

Isha Tomar
Sensor cleaning, rolling features, ML baselines

Yashi Agrawal
Report: dataset and EDA, models, references

Mohammad Affan Anas
Data download, setup guide, feature-engineering report

github.com/vks-g/cmapss-rul-hybrid

Where this work fits

Jardine 2006 · Si 2011 · Lei 2018

Prognostics and RUL

Methods are physics-based, data-driven or hybrid. Data-driven pipelines end in health-stage division, hence our stage-wise errors.

Saxena 2008 · Ramasso & Saxena 2014

The C-MAPSS benchmark

Simulated run-to-failure engines and the asymmetric PHM08 score. 70+ papers were hard to compare: different subsets, targets, protocols.

Khelif 2017 · Breiman 2001 · Chen 2016 · Zou 2005

Feature-based ML

Direct regression from sensor summaries to RUL. Our baselines: random forest, XGBoost, elastic net.

Babu 2016 · Zheng 2017 · Li 2018

Sequence models and the gap

CNNs and LSTMs on raw windows report lower errors, often with a capped target. Phase 2 tests this under our fixed protocol.

The theory in four formulas

Regime z-score

$$z = (x - \mu_r) \div \sigma_r$$

μ_r, σ_r: the regime's mean and standard deviation, from training rows only.

Rolling slope

$$b = (n\Sigma tx - \Sigma t\Sigma x) \div (n\Sigma t^2 - (\Sigma t)^2)$$

Least-squares slope of sensor x against cycle t over the last $n \le w$ cycles.

Elastic Net loss

$$(1/2N)\Sigma(y-\hat{y})^2 + \alpha\rho\Sigma|\beta| + \tfrac{1}{2}\alpha(1-\rho)\Sigma\beta^2$$

α: penalty strength. ρ: L1 ratio. L1 zeroes weights; L2 shrinks correlated ones together.

NASA score

$$s = \Sigma\ e^{(d/10)-1}\ (d>0)\ \text{or}\ e^{(-d/13)-1}$$

d = predicted − true RUL. Late errors (d > 0) cost more than early ones.

All 155 features

Context · 5

cycle
op_1 · op_2 · op_3
regime

Sensors · 15

s_2 · s_3 · s_4
s_6 · s_7 · s_8
s_9 · s_11 · s_12
s_13 · s_14 · s_15
s_17 · s_20 · s_21

Rolling · 135

15 sensors
× mean · slope · health
× windows 5 · 10 · 20

e.g. s_11_health_5

Spearman ρ with RUL

cycle -0.79
s_4_mean_5 -0.75
s_11_mean_5 -0.75
s_4_mean_10 -0.75
s_17_mean_10 -0.75

Training rows; descriptive only.

Dropped in FD001: s_1, s_5, s_10, s_16, s_18, s_19 (constant). Raw comparison set: cycle, 3 settings, all 21 sensors = 25 predictors.

Reproducible and tested

99 unit tests

Loading, labels, splits, cleaning, regimes, rolling, metrics, models

Seeded and engine-safe

Seed 42 everywhere; splits assert no engine is on both sides

Notebooks 01 → 04

Restart and run all, from the raw NASA files

One-command data

scripts/download_data.py fetches the 12 files

Reviewed pull requests

Every change on its own branch, reviewed into dev

```
vks-g/cmapss-rul-hybrid
rul/data          load · labels · split
rul/features      cleaning · regimes · rolling · matrix
rul/models        ml · experiment
rul/evaluation    metrics · official · plots
tests             13 files · 99 tests
notebooks         phase1/01 → 04
results           metrics JSON · per-engine CSV
```

One engine dominates the NASA score

ENGINEERED RANDOM FOREST · TEST ENGINE 1

Quantity	Value
Cycles observed	31
True RUL / predicted	112 / 230
Error (late)	+118 cycles
Penalty e^(11.8) − 1	133,652
Other 99 engines	11,758
Total NASA score	145,410

92%

The penalty grows exponentially, so one very late prediction outweighs the other 99 engines combined.

Why: a young, healthy-looking engine gets the long lives of training engines (128–362 cycles), but no test engine had more than 145 left.

So we read NASA together with RMSE, MAE and the stage table.

FD002: six operating regimes

TRAINING CYCLES PER ROUNDED op_1

op_1 band	Cycles
0	8,044
10	8,096
20	8,122
25	8,002
35	8,037
42	13,458

SPREAD OF REGIME MEANS (GLOBAL SD)

Sensor	Before	After
s_2	1.02	≈ 0
s_3	1.03	≈ 0
s_11	1.05	≈ 0

k-means with k = 6 on the three settings, fitted on training rows only. Afterwards the condition offset is gone; what is left is wear.

Capping RUL at 125 reverses raw vs engineered

RANDOM FOREST · TEST RMSE AGAINST THE TRUE, UNCAPPED RUL

Training target	Engineered	Raw
Uncapped (Phase 1 primary)	27.55	26.15
Capped at 125 (ablation)	16.21	18.66

Same grids, folds and seed as notebook 04; only the training target is clipped. Produced by scripts/capped_rul_ablation.py.

16.21

RMSE for engineered Random Forest with a capped target, against 18.66 with raw inputs.

A cap removes the impossible task of guessing how long a still-healthy engine will last. Whether 125 is the right cap has to be settled on training data in the Phase 2 ablation, not on the test set.

SOURCE: FD001_TEST_CAPPED125.JSON